컴시프 퀴즈 — 0910 (메모리 하이어라키 & 스택 버퍼 오버플로우)

범위: 슬라이드 05 (The Memory Hierarchy) 전체 + 슬라이드 06 (Stack Buffer Overflow) 전체 / 강의 녹취 0910_rev.txt **구성:** OX 5 · 서술형 5 · 예제 10 · 심화 3 (총 23문항)

> 답안은 문서 맨 끝 "정답 및 해설"에 모아두었습니다. 서술형·예제는 아래 빈칸에 직접 풀어보세요.

Part 1. OX (참/거짓)

각 명제가 참이면 O, 거짓이면 X. 거짓이면 어디가 틀렸는지 한 줄로 적으시오.

1-1. 메모리 하이어라키에서 레이어 k는 레이어 k+1의 캐시 역할을 하며, 하이어라키가 잘 동작하는 근본 이유는 프로그램의 **로컬리티(locality)** 때문이다. ( )

1-2. `temporal locality`는 최근 접근한 데이터를 곧 다시 접근할 가능성이 높다는 것이고, `spatial locality`는 접근한 주소 근처의 데이터를 곧 접근할 가능성이 높다는 것이다. 2차원 배열을 로우 메이저(row-major)로 저장하면서 열(column) 우선으로 순회하면 spatial locality가 나빠진다. ( )

1-3. NX(non-executable stack) 보호가 켜져 있으면 스택에 주입한 셸코드로 점프해도 실행되지 않지만, **ROP는 이미 실행 가능한 영역(코드/라이브러리)에 있는 조각들을 재사용**하므로 NX만으로는 막지 못한다. ( )

1-4. 스택 카나리(stack canary)는 `main` 같은 caller의 리턴 어드레스가 덮여 쓰였는지를 함수 **진입(prologue) 직후**에 검사한다. ( )

1-5. `ret` 인스트럭션의 머신 코드는 단일 바이트 `0xc3`이며, "리턴으로 끝나는 인스트럭션 시퀀스"를 가젯(gadget)이라 부른다. ( )

Part 2. 서술형

문장 몇 개로 충분히 설명하시오 (각 5~8문장 권장).

2-1. 캐시(cache)를 "스테이징 에어리어(staging area)"라고 부르는 이유를 메모리 하이어라키의 관점에서 설명하시오. k 레이어와 k+1 레이어의 관계, 히트/미스, 그리고 이 구조가 "왜 동작하는가"(로컬리티와의 연결)까지 포함하시오.

2-2. 프로그램이 잘못된 메모리를 접근했을 때 화면에 `Segmentation fault`가 뜨기까지, **CPU → (MMU/페이지 테이블) → 커널 → 유저 프로세스**로 이어지는 일련의 사건(chain of action)을 순서대로 서술하시오. 교수가 "이 체인을 이해하는 게 수업 목표"라고 강조한 부분이다.

CPU가 페이지테이블을 룩업해서 (?) 메모리에 접근하려고 할 때 잘못됐다는 걸 알고 커널을 통해 유저 프로세스에 에러를 레이즈한다?

2-3. `copy_name(char *src)` 안에서 `char name[32]`에 `strcpy(name, src)`를 하는 코드가 있다. 사용자가 32바이트를 넘겨도 곧바로 죽지 않을 수 있고, 어느 지점을 넘어가면 죽는다. **스택 프레임의 배치(name 버퍼 / saved %rbp / 리턴 어드레스)를 근거로**, 왜 32바이트 직후에 반드시 죽는 것은 아닌지, 그리고 무엇을 덮어써야 "죽는" 또는 "공격이 되는" 상황이 되는지 설명하시오.

2-4. 버퍼 오버플로우에 대한 세 가지 방어 기법 — **(1) 안전한 함수 사용, (2) ASLR, (3) NX(non-executable), (4) 스택 카나리** 중 세 가지를 골라 — 각각이 "공격의 어느 단계를 무력화하는가"를 기준으로 비교하시오. 특히 ASLR과 NX가 각각 무엇을 어렵게 만드는지 구분하시오.

2-5. ROP(Return-Oriented Programming)가 왜 등장했으며, 코드 인젝션(code injection) 공격과 근본적으로 무엇이 다른지 설명하시오. 가젯의 정의, 가젯을 스택에 어떻게 배열하는지, `ret`가 어떻게 다음 가젯으로 실행을 이어주는지를 포함하시오.

Part 3. 예제

AT&T 문법, x86-64 System V 호출 규약 (인자: `%rdi %rsi %rdx %rcx %r8 %r9`, 반환: `%rax`).

3-1. 다음 두 순회 코드 중 배열 `a`에 대한 로컬리티(특히 spatial locality)가 더 좋은 것은 무엇이며 그 이유는? `M`, `N`은 충분히 크다고 가정한다.

```
/* (A) */
int s=0;
for(int i=0;i<M;i++)
  for(int j=0;j<N;j++)
    s += a[i][j];
```

```
/* (B) */
int s=0;
for(int j=0;j<N;j++)
  for(int i=0;i<M;i++)
    s += a[i][j];
```

아니 당연히 (A)지, 근데 이게 row major로 저장되었다고 확언할 수 있나? 애초에 이 자료형이 row major로 저장된건가?

3-2. 아래는 `-fno-stack-protector`로 컴파일한 `copy_name`의 디스어셈블리다 (버퍼 크기 하나만 원본과 다르다).

```
copy_name:
  push   %rbp
  mov    %rsp,%rbp
  sub    $0x40,%rsp
  mov    %rdi,-0x38(%rbp)
  lea    -0x30(%rbp),%rax
  mov    -0x38(%rbp),%rdx
  mov    %rdx,%rsi
  mov    %rax,%rdi
  call   strcpy
  ...
  leave
  ret
```

a. `name` 버퍼의 시작 주소는 `%rbp`를 기준으로 어디인가?

b. `name` 버퍼의 시작에서 **saved %rbp**까지, 그리고 **리턴 어드레스**까지 각각 몇 바이트 떨어져 있는가?

c. 따라서 리턴 어드레스를 덮으려면 최소 몇 바이트를 입력해야 하는가?

return 도 덮으려면 64byte 필요한가?

3-3. 슬라이드에 나온, 카나리가 켜진(`-fstack-protector`) `copy_name`의 앞뒤 코드다.

```
  mov    %fs:0x28,%rax
```

```
  mov    %rax,-0x8(%rbp)
  xor    %eax,%eax
  ...                       # strcpy 등
  mov    -0x8(%rbp),%rax
  sub    %fs:0x28,%rax
  je     .Lok
  call   __stack_chk_fail
.Lok:
  leave
  ret
```

a. `%fs:0x28` 은 무엇이며 어떤 성질을 가지는가?

b. `sub %fs:0x28,%rax` 다음 `je` 가 취하는(taken) 조건을 플래그로 설명하시오. (힌트: 어떤 플래그가 1이어야 하나?)

c. 공격자가 리턴 어드레스만 덮고 카나리를 그대로 두는 것이 왜 어려운가?

3-4. 다음 어셈블리를 C로 복원하시오. 인자는 `%rdi` (부호 있는 `long x`), 반환은 `%rax`.

```
f:
  movl   $0, %eax
.L2:
  testq  %rdi, %rdi
  je     .L3
  movq   %rdi, %rdx
  andq   $1, %rdx
  addq   %rdx, %rax
  sarq   %rdi
  jmp    .L2
.L3:
  ret
```

(주의: `sarq` 인 점이 함정이다. `%rdi` 가 음수이면 어떤 일이 벌어지는가?)

3-5. ROP 개념 예제 (슬라이드 06-34 변형). 공격자가 `%rip` (다음에 실행할 주소)와 스택 내용을 원하는 대로 채울 수 있다. 목표: **상수 `C` 를 메모리 주소 `A` 에 저장(`*A = C`)**. 가젯이 다음뿐이라고 하자.

```
G1:  pop %rax ; ret
G2:  pop %rbx ; ret
G3:  mov %rbx, (%rax) ; ret
```

`%rip` 가 처음 가리킬 곳부터 시작해서, 스택에 어떤 값들을 위→아래(리턴이 소비하는 순서)로 배치해야 `*A = C` 가 실행되는지 쓰시오.

3-6. ROP 체인 (슬라이드 06-35 변형). 이제 `%rax` , `%rbx` 는 임의 초기값이고 공격자는 **`%rip` 와 스택만** 통제한다. 3-5와 같은 가젯 `G1(pop %rax;ret)` , `G2(pop %rbx;ret)` , `G3(mov %rbx,(%rax);ret)` 을 사용해 `*A = C` 를 수행하는 스택 배치를 순서대로 쓰고, **각 `ret` 가 실행될 때마다 `%rsp` 가 어디를 가리키는지** 3~4단계 추적하시오.

3-7. 아래는 어떤 상수 곱셈을 하는 최적화 코드다. `%rdi = x`(long)일 때 `%rax`에 무엇이 들어가는가? 식으로 유도하시오.

```
  leaq   (%rdi,%rdi,4), %rax     # rax = ?
  leaq   (%rax,%rax,2), %rax     # rax = ?
  salq   $3, %rax                # rax = ?
```

3-8. "언제 죽는가" 실험. `name[32]` 버퍼에서 saved %rbp가 `name` 시작으로부터 32바이트 위, 리턴 어드레스가 40바이트 위에 있다고 하자. 입력 문자열 길이(널 종료 포함 실제 기록 바이트)가 다음일 때 각각 무엇이 덮이는지 표를 채우시오. `strcpy`는 소스의 널 문자까지 복사한다.

입력 (문자수)	기록 바이트(널 포함)	덮이는 대상
31	32	(a)
32	33	(b)
40	41	(c)
48	49	(d)

3-9. 다음 중 **unintended gadget**(의도치 않은 가젯)이 x86에서 생기는 이유를 가장 잘 보여주는 것을 고르고 설명하시오. 바이트열 `f7 c7 07 00 00 00 0f 95 45 c3`은 정렬된 시작(offset 0)에서는 `test` / `setne`로 디코드되지만, **중간 바이트(offset 3의 `c3`을 포함한 뒷부분)**를 새 시작점으로 잡으면 `... ret`로 끝나는 전혀 다른 시퀀스가 된다. (a) x86 명령어의 어떤 성질 때문에 이런 일이 가능한가? (b) 이 성질이 ROP 가젯 확보에 왜 유리한가?

3-10. 틀린 코드 찾기. 의도한 C는 `unsigned long`을 오른쪽으로 1비트 논리 시프트(`x >>= 1`)하는 것인데, 컴파일된 결과가 아래와 같다. 어느 줄이 잘못됐고 어떻게 고쳐야 하는가?

```
shift_one:
  movq   %rdi, %rax
  sarq   %rax          # <-- ?
  ret
```

Part 4. 심화 (흔한 함정)

4-1. (OX + 서술) "NX(DEP)가 켜져 있으면 스택 오버플로우로는 아무 코드도 실행시킬 수 없다." — 참/거짓? 거짓이라면 어떤 공격 기법이 이 전제를 무너뜨리는지, 그 기법이 NX를 어떻게 우회하는지 설명하시오.

4-2. (개념 함정) 스택 카나리가 있어도 뚫리는 경우가 있다. 다음 세 가지 중 **카나리로 막지 못하는 시나리오**를 모두 고르고 이유를 쓰시오. - (ㄱ) 버퍼 바로 뒤(카나리보다 앞쪽, 더 낮은 주소)에 있는 다른 지역 변수 / 함수 포인터를 덮어쓰는 경우 - (ㄴ) 포맷 스트링 등으로 카나리 값을 **읽어낸(leak)** 뒤, 그 값을 그대로 다시 써넣으며 리턴 어드레스까지 덮는 경우 - (ㄷ) 버퍼를 넘겨 카나리 → saved %rbp → 리턴 어드레스를 통째로 연속으로 덮어쓰는 경우

4-3. (ASLR 함정) “ASLR이 켜져 있으면 `print_passwd` 같은 함수의 주소를 절대 알 수 없어 ROP도 불가능하다.” 이 주장이 잘못된 이유를 설명하시오. (힌트: PIE가 아닌 실행 파일의 코드 세그먼트, 정보 유출(leak)로 얻은 베이스 주소 + 오프셋 계산, 고정 매핑 영역 등을 고려하라.)

정답 및 해설

Part 1

1-1. O. 각 레이어 k는 바로 아래(더 크고 느린) 레이어 k+1의 스테이징 에어리어(캐시)이고, 대부분의 접근이 로컬리티 덕분에 k에서 끝나므로 하이어라키가 효과를 낸다.

1-2. O. 로우 메이저 저장 + 열 우선 순회는 메모리를 큰 보폭(stride N)으로 건너뛰며 접근하므로 같은 캐시 라인/블록을 재활용하지 못해 spatial locality가 나빠진다.

1-3. O. NX는 "주입한 데이터 영역의 코드 실행"을 막는다. ROP는 새 코드를 주입하지 않고 이미 실행 가능(executable)한 코드의 가젯을 재사용하므로 NX를 우회한다.

1-4. X. 카나리 검사는 함수 **리턴 직전(epilogue)** 에 한다. 진입 시에는 카나리를 **심어두기만** 하고(`mov %fs:0x28,%rax; mov %rax,-0x8(%rbp)`), 리턴 직전에 값이 바뀌었는지 확인한다(`sub %fs:0x28,%rax; je ...`). 값을 "쓰기 직전"에 확인해야 의미가 있기 때문이다.

1-5. O. `ret` = `0xc3` (1바이트). 가젯 = ret로 끝나는 짧은 인스트럭션 시퀀스. (강의 녹취에서 교수가 "0xc2"라고 말한 것은 착오이며 슬라이드 06-32는 0xc3.)

Part 2

2-1. 캐시가 스테이징 에어리어인 이유:

- 각 레이어 k는 자기보다 크고 느린 k+1의 **일부를 담아두는** 더 작고 빠른 저장소다.
- CPU가 데이터를 원하면 먼저 k를 본다 — 있으면 **히트**로 빠르게 끝나고, 없으면 **미스**로 k+1에서 블록 단위로 가져와 k에 채운다.
- 즉 k+1로 "가기 전에 반드시 거쳐 가는 무대(staging)" 역할이다.
- 이 구조가 이득인 이유는 **로컬리티**: 대부분의 접근이 최근/근처 데이터라 k에서 해결되고, 느린 k+1로 내려갈 일이 드물다.
- 그래서 전체 평균 접근 시간이 (가장 빠른 레이어에 가깝게) 줄어든다.

2-2. Segfault chain:

1. CPU가 인스트럭션을 실행하며 어떤 가상 주소에 접근한다.
2. MMU가 그 가상 주소를 페이지 테이블로 변환(워크)하려 한다 — 매핑이 없거나 권한(R/W/X)이 안 맞으면 유효하지 않은 접근이다.
3. CPU가 이를 감지해 **익셉션(page fault 등)** 을 발생시킨다.
4. 제어가 **커널의 익셉션 핸들러**로 넘어간다.
5. 핸들러가 "정당한 접근이 아니다"라고 판정하면 해당 유저 프로세스에 **시그널(SIGSEGV)** 을 보낸다.
6. 유저 프로세스가 죽으면서 `Segmentation fault`가 표시된다.

2-3. 32바이트 직후에 반드시 죽지 않는 이유:

- 스택 프레임은 낮은 주소부터 `name[32]` → (패딩) → saved %rbp → 리턴 어드레스 순으로 놓인다.
- 32바이트를 조금 넘겨도 처음엔 패딩/saved %rbp를 건드릴 뿐, 그 즉시 크래시가 나는 건 아니다.
- saved %rbp가 망가지면 caller로 돌아가 caller가 리턴할 때 프레임 포인터가 이상해져 **나중에** 문제가 될 수 있다(즉시 죽지 않을 수 있음).
- 실제로 "죽는" 것은 대개 **리턴 어드레스**가 깨져 `ret`가 엉뚱한/무효한 주소로 점프할 때다.
- 정확히 몇 바이트에서 죽는지는 버퍼 크기·패딩·컴파일러에 따라 달라진다.
- 공격이 성립하려면 크래시가 아니라 **리턴 어드레스를 의도한 주소로** 덮어써 `%rip`를 통제해야 한다.

2-4. 방어 기법 비교 (공격 단계별):

- **안전한 함수(fgets/strncpy)**: 애초에 **오버플로우가 일어나는 단계**를 없앤다(경계 검사). 단 크기 인자를 잘못 주면 여전히 취약.
- **ASLR**: 오버플로우가 나도, 공격자가 **점프할 주소를 알아내는 단계**를 어렵게 한다(코드/스택/라이브러리 베이스를 매 실행 랜덤화).
- **NX**: 스택 등 데이터 영역에 **주입한 코드를 실행하는 단계**를 막는다(그 영역을 실행

불가로 표시).

- **카나리**: 리턴 어드레스가 **덮였는지 탐지하는 단계**를 추가한다(리턴 직전 변조 검사).
- 요약: ASLR은 "어디로 점프할지 모르게", NX는 "주입 코드는 실행 못 하게", 카나리는 "덮였으면 들킨다".

2-5. ROP:

- NX 때문에 **주입 코드 실행(code injection)** 이 막히자, 이미 있는 코드를 재사용하는 방향으로 등장했다.
- 코드 인젝션은 공격자가 **새 머신 코드**를 데이터 영역에 넣고 거기로 점프한다 — NX면 실패.
- ROP는 새 코드를 넣지 않고, 프로그램/라이브러리 안에 이미 존재하는 **가젯**(ret로 끝나는 짧은 시퀀스)만 사용한다.
- 스택에 **가젯 주소들을 연속으로 배열**해 둔다.
- 각 가젯의 `ret`가 스택 top의 다음 주소를 pop해 `%rip`에 넣으므로, 자동으로 다음 가젯으로 연결된다(체이닝).
- 작은 가젯들을 레고처럼 조합해 임의의 로직을 수행한다.

Part 3

3-1. (A) 가 좋다. 배열은 로우 메이저라 `a[i][0], a[i][1], ...`가 메모리에 연속으로 놓인다. (A)는 안쪽 루프가 `j`를 증가시켜 **연속 주소**를 순회하므로 spatial locality가 좋다(같은 캐시 블록 재사용). (B)는 안쪽 루프가 `i`를 증가시켜 매번 한 행(`stride = N*sizeof(int)`)씩 건너뛰므로 블록을 재활용 못 해 미스가 잦다.

3-2. - (a) `lea -0x30(%rbp),%rax` → `%rdi` (strcpy 1번째 인자=dest)로 넘어가므로 `name` 시작 = **`%rbp - 0x30`** (= -48). - (b) saved %rbp는 `%rbp`가 가리키는 곳(오프셋 0) → name 시작에서 **0x30(48)바이트** 위. 리턴 어드레스는 `%rbp + 8` → name 시작에서 **0x38(56)바이트** 위. - (c) 리턴 어드레스를 덮기 시작하려면 앞의 56바이트를 채운 뒤 1바이트 이상 → **최소 57바이트**(리턴 어드레스 8바이트를 온전히 덮으려면 56 + 8 = 64바이트).

3-3. - (a) `%fs:0x28`은 TLS(thread-local) 영역에 있는 **글로벌 카나리** 저장 위치로, 프로그램 로드 시 랜덤 값으로 초기화되는 읽기 전용 성격의 값이다. 함수는 진입 시 이 값을 스택(`-0x8(%rbp)`)에 복사해 둔다. - (b) `sub %fs:0x28,%rax`는 `%rax`(스택에 저장해둔 카나리) − (원본 카나리)를 계산한다. 두 값이 같으면 결과가 0 → **ZF=1**. `je`는 **ZF=1일 때 taken** → 정상(변조 없음)이면 `.Lok`으로 간다. 다르면(ZF=0) `__stack_chk_fail` 호출. - (c) 스택에서 버퍼는 카나리보다 **낮은 주소**에 있고, 리턴 어드레스는 카나리보다 **높은 주소**에 있다. 연속으로 덮어쓰는 `strcpy` 오버플로우는 리턴 어드레스에 닿기 전에 **반드시 카나리를 지나가며 덮게** 된다. 공격자가 원래 카나리 값을 모르면 변조가 검출되어 리턴 전에 죽는다.

3-4.

```
        long f(long x){
            long count = 0;
            while (x != 0){
                count += (x & 1);
                x = x >> 1;   /* 산술 시프트 (sarq) */
            }
            return count;
        }
```

- `testq %rdi,%rdi; je` → `x == 0`이면 종료.
- `andq $1` → 최하위 비트, `addq` → 누적, `sarq %rdi` → 부호 유지 우측 시프트.
- **함정**: `sarq`(산술)는 음수의 부호 비트를 채우며 내려가므로, `x`가 음수면 최상위 비트가 계속 1로 채워져 `x`가 절대 0이 되지 않는다 → **음수 입력 시 무한 루프**. (population count를 의도했다면 `shrq`(논리 시프트)를 써야 하고 인자도 `unsigned`여야 한다.)

3-5. `*A = C`, 초기부터 통제 가능. `%rip`가 G1을 가리키게 하고 스택을 이렇게 배치(리턴이 소비하는 순서, 위=먼저 소비):

```
[ G1 주소 ]   ← %rip 시작 (pop %rax; ret)
[ A       ]   ← pop %rax 가 가져감  → %rax = A
```

```
[ G2 주소 ]   ← 그 ret 가 점프    (pop %rbx; ret)
[ C       ]   ← pop %rbx 가 가져감  → %rbx = C
[ G3 주소 ]   ← 그 ret 가 점프    (mov %rbx,(%rax); ret) → *A = C
```

실행: G1이 A를 %rax로, G2가 C를 %rbx로, G3가 `mov %rbx,(%rax)`로 `*A = C` 수행.

3-6. 3-5와 동일한 스택 배치를 쓴다(이 가젯 조합에서는 %rax/%rbx 초기값과 무관하게 pop으로 덮어쓰므로 3-5 배치가 그대로 정답이다). `%rsp` 추적:

```
초기: %rip=G1, %rsp → [A] (G1 주소는 이미 소비되어 %rip에 들어간 상태)
① G1: pop %rax  → %rax=A,  %rsp → [G2주소]
     ret         → %rip=G2, %rsp → [C]
② G2: pop %rbx  → %rbx=C,  %rsp → [G3주소]
     ret         → %rip=G3, %rsp → [그 다음]
③ G3: mov %rbx,(%rax) → *A=C
     ret         → %rip = 스택 다음 값 (다음 가젯으로 계속 체이닝 가
능)
```

핵심: 각 `pop`은 %rsp가 가리키는 값을 레지스터로 가져오며 %rsp를 8 증가시키고, 각 `ret`은 %rsp의 값을 %rip로 pop하며 다시 8 증가시킨다. 그래서 스택에 늘어놓은 순서대로 자동 연결된다.

3-7. - `leaq (%rdi,%rdi,4),%rax` → `%rax = x + 4x = 5x` - `leaq (%rax,%rax,2),%rax` → `%rax = 5x + 2·5x = 15x` - `salq $3,%rax` → `%rax = 15x << 3 = 15x·8 = 120x` → **`%rax = 120·x`**. (`leaq`는 주소 계산 형식을 산술에 빌려 쓸 뿐 메모리를 읽지 않는다.)

3-8. name[0..31]=32B, [32..39]=saved %rbp(8B), [40..47]=리턴 어드레스(8B). - (a) 31자→33B 기록: name(32) + 널 1B가 오프셋 32(=saved %rbp 첫 바이트) → 사실상 name만 꽉 채우고 널이 saved %rbp 1바이트 침범. **버퍼 경계선(대개 안전)**. ※ 표의 "31 / 32B"는 name 32칸을 딱 채움(널 포함) → **name 버퍼만** 덮음. - (b) 32자→33B: name 32칸 + 널 1B가 오프셋 32 → **saved %rbp의 최하위 1바이트** 침범 시작. - (c) 40자→41B: 오프셋 0~40 기록 → name(32) + saved %rbp(32~39, 8B) 전부 + 오프셋 40의 널 1B → **리턴 어드레스 최하위 1바이트** 침범. - (d) 48자→49B: 오프셋 0~48 → name + saved %rbp + 리턴 어드레스(40~47) 전부 + 오프셋 48 널 → **리턴 어드레스 전체를 덮음**(이때부터 `%rip` 통제 가능).

3-9. - (a) x86은 **가변 길이 명령어(variable-length instruction) + 정렬 강제 없음**. 어떤 바이트든 명령어의 새 시작점이 될 수 있어, 원래 의도한 스트림과 전혀 다른 명령열로 디코드될 수 있다. 예의 바이트열은 offset 0에서는 `test/setne` 지만 중간 바이트를 시작점으로 잡으면 `movl.../xchg/inc/ret`처럼 `ret`로 끝나는 다른 시퀀스가 나온다. - (b) 그래서 프로그래머가 "가젯으로 쓰라고 둔" 코드가 아니어도, 바이너리 곳곳의 바이트 조합에서 **의도치 않은 가젯**이 대량으로 나온다 → 공격자가 필요한 조각을 찾기 쉬워진다(라이브러리에서 수만 개 발견 가능).

3-10. `sarq %rax`가 잘못됐다. `unsigned long`의 `>>`는 **논리 시프트**여야 하므로 최상위에 0을 채우는 **`shrq %rax`**로 고쳐야 한다. `sarq`(산술)는 부호 비트를 채우므로 최상위 비트가 1인 큰 unsigned 값에서 틀린 결과가 나온다.

Part 4

4-1. 거짓. NX(DEP)는 "주입한 데이터 영역의 코드 실행"만 막는다. **ROP**(그리고 return-to-libc)는 새 코드를 주입하지 않고 이미 실행 가능한 영역의 가젯/함수를 재사용하므로 NX 아래에서도 임의 로직을 수행할 수 있다. 즉 NX는 code injection은 막지만 code reuse는 막지 못한다.

4-2. (ㄱ)과 (ㄴ). - (ㄱ) 카나리는 버퍼와 **리턴 어드레스 사이**에 있다. 버퍼보다 앞(더 낮은 주소)에 있는 다른 지역 변수나 함수 포인터를 덮는 공격은 카나리를 건드리지 않으므로 검출되지 않는다. (구조체 안에서 버퍼 뒤에 놓인 함수 포인터도 마찬가지 — 카나리로 못 막는 고전 사례.) - (ㄴ) 카나리 값을 **유출(leak)** 해 알아낸 뒤 그대로 다시 써넣으면, 검사 시점에 값이 "정상"이라 통과된다 → 리턴 어드레스를 덮어도 검출 실패. - (ㄷ)은 **막힌다**: 연속 덮어쓰기는 리턴 어드레스에 닿기 전 카나리를 (알 수 없는 값으로) 바꾸게 되어 리턴 직전 검사에서 걸린다.

4-3. 지나친 이유: - **비-PIE 실행 파일**: 실행 파일 본체(코드/PLT 등)가 고정 주소에 매핑되면 그 안의 함수·가젯 주소는 ASLR과 무관하게 알 수 있다(라이브러리만 랜덤일 수 있음). - **정보 유출(leak)**: 하나의 주소만 새어 나가면(예: libc 함수 주소) 라이브러리 **베이스 = 유출값 − 알려진 오프셋**을 계산해 나머지 전부를 복원할 수 있다. - **엔트로피/부

분 덮어쓰기: 32비트 등에서는 엔트로피가 낮아 브루트포스가 현실적이고, 페이지 정렬 때문에 하위 바이트는 고정이라 부분 덮어쓰기가 가능하다. - 결론: ASLR은 공격을 **어렵게** 만들 뿐 원천 차단이 아니며, 보통 정보 유출과 결합되면 우회된다.